

Documentation: Connect-4 Game

Mini-Max Algorithm with Alpha-Beta Pruning

AI Lab — Assignment 5

Uttam Mahata (2022CSB104)

March 24, 2026

Contents

1	Introduction	2
2	Problem Formulation	2
2.1	State Space	2
2.2	Actions	2
2.3	Terminal States	3
2.4	Win Condition	3
3	Winning Strategies	3
3.1	Strategy (a): Middle Column Placement	3
3.2	Strategy (b): Trapping / Blocking Opponents	3
3.3	Strategy (c): “7” Formation (Multi-directional Threats)	4
4	Mini-Max Algorithm	4
4.1	Overview	4
4.2	Evaluation Function	4
4.3	Recursive Definition	4
4.4	Algorithm Pseudocode	5
5	Alpha-Beta Pruning	6
5.1	Motivation	6
5.2	Cut-off Conditions	6
5.3	Example Cut-off	6
6	Implementation	6
6.1	Module Structure	6
6.2	Board Representation	7
6.3	Win Detection	7
6.4	Heuristic Evaluation	8
6.5	Core Mini-Max (Maximising Branch)	8
7	Example Game State	9
7.1	Board Notation	9
7.2	Heuristic Trace	9

8	Complexity Analysis	10
9	Edge Cases	10
10	Usage	10
11	Conclusion	11

1 Introduction

Connect-4 is a classic two-player strategy game played on a **6-row** $\times$ **7-column** vertical grid. Players alternate dropping coloured discs into columns; each disc falls under gravity to the lowest available row in that column. The first player to form a line of **four consecutive discs** (horizontally, vertically, or diagonally) wins.

This implementation pits a **human player** against a **computer AI** that uses the **Mini-Max algorithm with alpha-beta pruning** to determine the optimal move at every turn.

Key features of this implementation:

- Full 6 $\times$ 7 Connect-4 game with gravity-based disc placement.
- Mini-Max search to depth 5 with alpha-beta pruning for efficiency.
- Heuristic evaluation function encoding three winning strategies (centre preference, opponent blocking, multi-directional threats).
- Interactive console interface with ASCII board visualisation.
- Dual output: terminal display and `output.txt` game log.

2 Problem Formulation

2.1 State Space

A game state is a complete snapshot of the board at any point in time. Formally:

$$\mathcal{S} = \{0, 1, 2\}^{6 \times 7}$$

where each cell value is:

- 0 (EMPTY) — no disc placed.
- 1 (PLAYER) — human disc (**O**).
- 2 (AI) — computer disc (**X**).

The board is stored as a 2-D NumPy array of shape (6, 7). Row 0 is the *top*; row 5 is the *bottom*. Gravity is simulated by always inserting a disc into the lowest empty row of the chosen column.

2.2 Actions

At any turn, the set of legal actions is:

$$\mathcal{A}(s) = \{c \in \{0, \dots, 6\} \mid s[0][c] = \text{EMPTY}\}$$

i.e. all columns that still have at least one empty cell.

2.3 Terminal States

A state s is terminal if any of the following hold:

1. The AI has four discs in a row (AI wins).
2. The human has four discs in a row (human wins).
3. All 42 cells are occupied (draw).

2.4 Win Condition

Four consecutive discs of the same player in any of the following directions constitutes a win:

Direction	Window pattern
Horizontal	$(r, c), (r, c+1), (r, c+2), (r, c+3)$
Vertical	$(r, c), (r+1, c), (r+2, c), (r+3, c)$
Diagonal ↗	$(r, c), (r+1, c+1), (r+2, c+2), (r+3, c+3)$
Diagonal ↘	$(r, c), (r-1, c+1), (r-2, c+2), (r-3, c+3)$

3 Winning Strategies

Three strategies from the assignment specification are encoded directly into the AI's heuristic function.

3.1 Strategy (a): Middle Column Placement

The centre column (column 3) provides the maximum number of potential four-in-a-row connections — up to five different winning lines pass through it. The heuristic awards **+3 points** for every AI disc in column 3:

```
1 center_col = list(board[:, COLS // 2])    # column 3
2 score += center_col.count(piece) * 3
```

Listing 1: Centre column bonus in score_position.

This causes the AI to prefer column 3 on the opening move.

3.2 Strategy (b): Trapping / Blocking Opponents

A window containing *three opponent discs and one empty cell* is an immediate threat. The heuristic applies a **−4 penalty** to such windows, forcing the AI to prioritise blocking:

```
1 elif opp_count == 3 and empty == 1:
2     return -4
```

Listing 2: Opponent threat penalty in score_window.

When the AI encounters a board where the human has three in a row, the negative score dominates and the AI's best response is to drop into the blocking cell.

3.3 Strategy (c): “7” Formation (Multi-directional Threats)

By evaluating *all* windows (horizontal, vertical, both diagonals) simultaneously and searching to depth 5, the Mini-Max tree naturally discovers positions where the AI threatens to win in multiple directions at once — equivalent to the “7” formation described in the assignment. No single window can be blocked without the opponent winning via another.

4 Mini-Max Algorithm

4.1 Overview

Mini-Max is a recursive *backtracking* algorithm for two-player zero-sum games. It models:

- The **maximising player** (AI) — tries to reach states with the highest heuristic score.
- The **minimising player** (human) — tries to reach states with the lowest heuristic score.

Both players are assumed to play *optimally*.

4.2 Evaluation Function

The heuristic $h(s)$ scores a board state s from the AI’s perspective. It examines every *window* of 4 consecutive cells across all directions:

$$h(s) = \underbrace{3 \cdot |\text{AI discs in col 3}|}_{\text{centre bonus}} + \sum_{\text{windows } w} \text{scoreWindow}(w, \text{AI}) \quad (1)$$

Window scoring (`score_window`):

Window contents	Score
4 AI discs	+100
3 AI discs + 1 empty	+5
2 AI discs + 2 empty	+2
3 opponent discs + 1 empty	−4
All other configurations	0

Table 1: Heuristic window scores.

4.3 Recursive Definition

Let d be the remaining search depth. The Mini-Max value is:

$$\text{minimax}(s, d, \alpha, \beta, \text{max}) = \begin{cases} +10^8 & \text{if AI wins in } s \\ -10^8 & \text{if human wins in } s \\ h(s) & \text{if } d = 0 \text{ or board full} \\ \max_{a \in \mathcal{A}(s)} \text{minimax}(s_a, d-1, \alpha, \beta, \text{False}) & \text{if } \text{max} = \text{True} \\ \min_{a \in \mathcal{A}(s)} \text{minimax}(s_a, d-1, \alpha, \beta, \text{True}) & \text{if } \text{max} = \text{False} \end{cases} \quad (2)$$

where s_a is the board state after action a is applied.

4.4 Algorithm Pseudocode

```

1 Input:  board s, depth d, alpha  $\alpha$ , beta  $\beta$ , maximising flag
2 Output: (best_column, score)
3
4 minimax(s, d,  $\alpha$ ,  $\beta$ , maximising):
5     if AI wins(s):      return (None, +1e8)
6     if human wins(s):   return (None, -1e8)
7     if d=0 or full(s):  return (None, h(s))
8
9     valid <- { c | s[0][c] = EMPTY }
10
11     if maximising:
12         value      <- -inf
13         best_col   <- random(valid)
14         for c in valid:
15             drop AI disc in column c -> s'
16             _, score <- minimax(s', d-1,  $\alpha$ ,  $\beta$ , False)
17             if score > value:
18                 value <- score ; best_col <- c
19              $\alpha$  <- max( $\alpha$ , value)
20             if  $\alpha$  >=  $\beta$ : break          --  $\beta$  cut-off
21
22     else:
23         value      <- +inf
24         best_col   <- random(valid)
25         for c in valid:
26             drop human disc in column c -> s'
27             _, score <- minimax(s', d-1,  $\alpha$ ,  $\beta$ , True)
28             if score < value:
29                 value <- score ; best_col <- c
30              $\beta$  <- min( $\beta$ , value)
31             if  $\alpha$  >=  $\beta$ : break          --  $\alpha$  cut-off
32
33     return (best_col, value)

```

Listing 3: Mini-Max with Alpha-Beta Pruning (pseudocode)

5 Alpha-Beta Pruning

5.1 Motivation

Pure Mini-Max with branching factor $b = 7$ and depth $d = 5$ visits up to $b^d = 7^5 = 16\,807$ leaf nodes per move. Alpha-beta pruning eliminates subtrees that cannot affect the final decision, reducing this in practice to $O(b^{d/2})$ — equivalent to doubling the effective search depth.

5.2 Cut-off Conditions

Two variables are maintained throughout the recursive search:

- α — the *best score the maximiser can guarantee* so far on the path from the root. Initialised to $-\infty$.
- β — the *best score the minimiser can guarantee* so far on the path from the root. Initialised to $+\infty$.

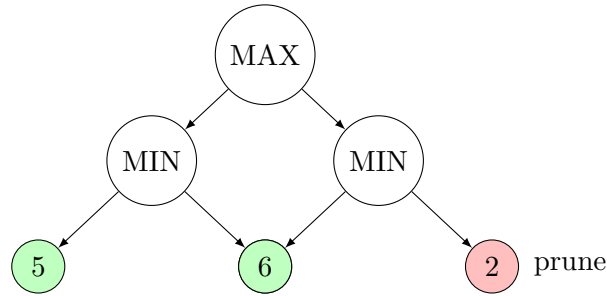
Beta cut-off (maximising node): if the current value $v \geq \beta$, the minimiser will never allow this branch — prune remaining siblings.

$$\alpha \leftarrow \max(\alpha, v) \quad \text{prune if } \alpha \geq \beta \quad (3)$$

Alpha cut-off (minimising node): if the current value $v \leq \alpha$, the maximiser will never follow this branch — prune remaining siblings.

$$\beta \leftarrow \min(\beta, v) \quad \text{prune if } \alpha \geq \beta \quad (4)$$

5.3 Example Cut-off



After the left MIN-node returns 3, $\alpha = 3$. In the right MIN-node, after seeing child value 2, $\beta = 2 \leq \alpha = 3$, so the remaining children are pruned.

6 Implementation

6.1 Module Structure

```

1 connect4.py
2 |- Constants (ROWS, COLS, EMPTY, PLAYER, AI, DEPTH)
3 |- Connect4 # game + AI encapsulated in
  one class
4 |   |- __init__() # initialise 6x7 numpy board
5 |   |- tee(*args) # print and log simultaneously
6 |   |- is_valid_column(col) # True if column has space
7 |   |- get_next_open_row(col) # lowest empty row (on self.
  board)
8 |   |- drop_piece(board, row, col, p) # place disc in-place
9 |   |- is_board_full() # check draw condition
10 |   |- winning_move(board, piece) # 4-in-a-row detection
11 |   |- score_window(window, piece) # heuristic for one window of
  4
12 |   |- score_position(board, piece) # full board heuristic
13 |   |- minimax(board, d, a, b, max) # Mini-Max with alpha-beta
14 |   |- _get_open_row(board, col) # helper (works on any board
  copy)
15 |   |- get_best_move() # calls minimax, returns best
  column
16 |   |- print_board() # ASCII visualisation
17 |   |- play() # main interactive game loop
18 |- __main__ # entry point

```

Listing 4: Module-level layout of connect4.py

6.2 Board Representation

```

1 self.board = np.zeros((ROWS, COLS), dtype=int)
2 # ROWS=6, COLS=7
3 # board[0][c] = top row (gravity fills from bottom up)
4 # board[5][c] = bottom row

```

Listing 5: Board initialisation.

6.3 Win Detection

```

1 def winning_move(self, board, piece):
2     # Horizontal
3     for r in range(ROWS):
4         for c in range(COLS - 3):
5             if all(board[r][c + i] == piece for i in range(4)):
6                 return True
7     # (vertical, diagonal-positive, diagonal-negative omitted for
  brevity)
8     return False

```

Listing 6: winning_move — horizontal check shown as example.

6.4 Heuristic Evaluation

```
1 def score_position(self, board, piece):
2     score = 0
3     # Strategy (a): centre column bonus
4     center_col = list(board[:, COLS // 2])
5     score += center_col.count(piece) * 3
6
7     # Horizontal windows
8     for r in range(ROWS):
9         row_arr = list(board[r, :])
10        for c in range(COLS - 3):
11            score += self.score_window(row_arr[c:c + 4], piece)
12
13    # Vertical windows
14    for c in range(COLS):
15        col_arr = list(board[:, c])
16        for r in range(ROWS - 3):
17            score += self.score_window(col_arr[r:r + 4], piece)
18
19    # Diagonal (positive and negative slopes)
20    for r in range(ROWS - 3):
21        for c in range(COLS - 3):
22            window = [board[r + i][c + i] for i in range(4)]
23            score += self.score_window(window, piece)
24    for r in range(3, ROWS):
25        for c in range(COLS - 3):
26            window = [board[r - i][c + i] for i in range(4)]
27            score += self.score_window(window, piece)
28    return score
```

Listing 7: score_position — full board evaluation.

6.5 Core Mini-Max (Maximising Branch)

```
1 if maximizing_player:
2     value = -math.inf
3     best_col = random.choice(valid_cols)
4
5     for col in valid_cols:
6         row = self._get_open_row(board, col)
7         b_copy = board.copy()
8         b_copy[row][col] = AI
9
10        _, new_score = self.minimax(b_copy, depth - 1,
11                                   alpha, beta, False)
12        if new_score > value:
13            value = new_score
14            best_col = col
15
16    alpha = max(alpha, value)
```

```

17         if alpha >= beta:                # beta cut-off
18             break
19
20     return best_col, value

```

Listing 8: Mini-Max maximising branch with beta cut-off.

7 Example Game State

7.1 Board Notation

During a game session the board is rendered as:

```

+---+---+---+---+---+---+---+
| . | . | . | . | . | . | . |
+---+---+---+---+---+---+---+
| . | . | . | . | . | . | . |
+---+---+---+---+---+---+---+
| . | . | . | X | . | . | . |
+---+---+---+---+---+---+---+
| . | . | . | X | . | . | . |
+---+---+---+---+---+---+---+
| . | . | . | X | O | . | . |
+---+---+---+---+---+---+---+
| . | . | . | O | O | O | . |
+---+---+---+---+---+---+---+
| . | . | . | O | O | O | . |
+---+---+---+---+---+---+---+
0   1   2   3   4   5   6

```

7.2 Heuristic Trace

For the board above, the AI (X) evaluates the position:

- Column 3 centre bonus: 3 discs $\times$ 3 = **+9**.
- Vertical window {X,X,X,.} in column 3 rows 2–5: 3 AI + 1 empty $\Rightarrow$ **+5**.
- Human window {O,O,O,.} in row 5 cols 3–6: 3 opp + 1 empty $\Rightarrow$ **-4**.

The blocking penalty (−4) signals the AI to drop into column 6 (or whichever cell completes the block) before extending its own vertical threat — this is Strategy (b) in action.

8 Complexity Analysis

Component	Time	Space
Naive Mini-Max (depth d , branching b)	$O(b^d)$	$O(bd)$
With alpha-beta pruning (best case)	$O(b^{d/2})$	$O(bd)$
With alpha-beta pruning (average case)	$O(b^{3d/4})$	$O(bd)$
<code>score_position</code> per call	$O(R \cdot C)$	$O(1)$
<code>winning_move</code> per call	$O(R \cdot C)$	$O(1)$
Per-move total ($b = 7$, $d = 5$)	$\approx 1,000 - 5,000$ nodes	$O(35)$

Table 2: Asymptotic complexity ($R = 6$ rows, $C = 7$ cols, $b = 7$, $d = 5$).

Alpha-beta pruning effectively reduces the search tree from $7^5 = 16\,807$ nodes to roughly $7^{2.5} \approx 1,297$ nodes in the best case, enabling depth-5 search to complete in well under a second per move.

9 Edge Cases

1. **Full column input:** The game loop re-prompts the human with an error message rather than accepting an invalid column.
2. **Non-integer input:** A `ValueError` is caught and the player is asked again.
3. **Out-of-range column:** Columns outside $[0, 6]$ are rejected before any board access.
4. **Draw (board full, no winner):** Detected after every move via `is_board_full()`.
5. **Depth-0 leaf with no terminal:** `score_position` is called to return a heuristic value without requiring a terminal state.
6. **Only one valid column remaining:** Mini-Max still evaluates it correctly; alpha-beta pruning simply has nothing to prune.

10 Usage

```

1 # Activate virtual environment (if using uv)
2 source .venv/bin/activate
3
4 # Run from the project root
5 python Assignment-5/connect4.py

```

Listing 9: Running the game.

Symbol	Meaning
.	Empty cell
O	Human disc
X	Computer (AI) disc

Enter a column number (0–6) when prompted. The AI responds immediately. A full game log is saved to `output.txt` in the current working directory after the session ends.

11 Conclusion

This implementation demonstrates the Mini-Max algorithm as an effective AI for Connect-

4. Key design decisions:

- **Alpha-beta pruning** — eliminates provably irrelevant subtrees, enabling depth-5 search with negligible latency.
- **Centre-column heuristic** — encodes domain knowledge that column 3 is the strategically strongest opening position.
- **Window-based evaluation** — the same `score_window` function uniformly handles all four directions, keeping the heuristic concise and consistent.
- **Threat-blocking penalty** — a -4 score for opponent three-in-a-row windows guarantees the AI blocks immediate threats before pursuing its own longer-term plans.
- **Configurable depth** — the `DEPTH` constant can be increased for stronger (slower) play or decreased for faster (weaker) play without any other code changes.